

VLA 내부 활성화에 나타나는 세밀 행동 단계 구조

박경태, 김상우, 오윤선[†]
한양대학교

rudxo1997@hanyang.ac.kr, [†] 교신저자 이메일

Fine-Grained Action-Phase Structure in the Internal Activations of a VLA Model

Kyungtae Park, Sangwoo Kim, Yoonseon Oh[†]
Hanyang Univ.

요약

Vision-Language-Action(VLA) 모델이 작업을 수행할 때 스스로 현재 어떤 작업 단계를 진행중인지를 우리가 알 수 있다면, 이는 VLA 모델에 대한 설명가능성과 추론 시 제어 및 개입의 출발점이 된다. 본 연구는 VLA의 내부 activation을 통해 모델이 스스로 인식하는 현재 작업 단계를 직접 읽는다. Gr00t N1.5의 action head(DiT)의 residual stream activation을 라벨 없이 clustering 하면, 사람이 정의한 primitive action phase보다 더 세밀한 action phase cluster 이 생긴다. 이때 한 action phase cluster는 특정 primitive action phase에 국한되어 나타나며 다른 primitive action phase에서는 잘 나타나지 않는 특성을 관찰했으며(순도 0.80), 단순 시간 진행도 대조군보다 1.67 bits 높은 단계 정보를 담는다. 이 구조는 데이터 처리 과정을 바꾸어도(AE→SAE) 같은 시점에서 phase 전환을 유지하고, 다양한 task, scene에서도 유지된다($z + 8.8 \sim +13.5$). 즉, VLA activation에는 세밀하지만 정합적인 action phase 구조가 존재하며 온라인 관독에 이를 활용 할 수 있으며, 본 논문에서 이를 실험적으로 확인하였다.

I 서론

많은 AI 모델들이 그러하듯 VLA 모델 또한 내부적으로 어떤 상태를 가지는지는 사람이 이해하기 힘든 블랙박스과 같다. 하지만 VLA 모델은 실제 존재하는 로봇을 조작하기 때문에 LLM 등과 달리 안전과 신뢰가 더욱 중요하다. 따라서 모델이 지금 무슨 작업을 진행중이라고 인지하는지 사람이 파악할 수 있어야 한다는 설명가능성(XAI)이 중요하다. 일반적으로 Pick & Place task에서는 reach → grasp → transport → place 같은 primitive action phase를 사용할 수 있다. 이러한 현재 action phase를 파악하는 방법에는 외부 관찰자가 카메라나 기타 센서들을 활용해 로봇의 현재 상태나 궤적같은 외부 관찰자를 활용하는 방법이 있다. 하지만 이러한 외부 관찰자로는 현재 로봇이 보기에 무엇을 하고 있는지를 알려줄 뿐, 현재 저 로봇을 움직이는 인공지능이 스스로 무슨 action을 진행중인지는 알려주지 않는다. 모델이 스스로 무엇을 하고 있다고 인지하는지, 즉 모델의 internal representation이 가리키는 action phase는 모델 내부의 activation을 직접 읽어야만 알 수 있다. 이런 내부 표상을 아는것은 길보기 행동과 어긋났을 경우에 수정을 진행하기위한 전제 조건이다. VLA 내부 표현을 해석하려는 최근 시도들은 Sparse Auto Encoder(SAE)로 residual stream에서 grasp·transport 같은 primitive action에 대응하는 feature를 찾아내거나, linear probe로 layer 별 상태를 관측하고 그 방향으로 activation steering을 적용해 행동을 제어 할 수 있음을 보였다. 그러나 이들은 사람이 rollout 영상을 검토해 발견 한 feature와 정렬해주어 이름을 붙일 뿐, interval representation이 사람이 정의한 action phase와 어떤 해상도로 대응하는지 정량화하지 않는다. VLA의 internal action phase를 추론중에 online으로 읽을 수 있으면 설명을 넘어 조건부 개입이나 실패감지의 조건신호로 활용 할 수 있다. 본 연구는 Gr00t N1.5가 RoboCasa 환경에서 주방 조작을 수행하는동안 activation을 Autoencoder와 clustering을 사용해 분석하고 모델의 internal action phase가 사람이 불린 action phase와 어떤 관계를 가지는지를 MI를 활용해 정량화 한다.

II 방법

Data RoboCasa 주방 조작 환경에서 GR00T N1.5의 rollout을 수집한다. 대상은 5개 task(OpenDrawer, PnPCounterToCab, OvenRack, DishwasherRack, CoffeeSetupMug)에 속하는 10개 instruction - 같은 task로 분류되더라도 실제 모델에 들어가는 language instruction이 다른 case가 있으며, 분석은 이 instruction 단위로 수행한다- 각instruction

별로 10 scene에서, scene별로 diffusion noise를 다르게 해 10회 반복하여 데이터를 수집하였다. 각 rollout의 매 스텝에서 action head(DiT)의 residual stream activation을 캡처한다(layer 12, denoising step 3(last), 49개 토큰 평균 → 스텝당 1536차원). 여기서 한 스텝은 정책 추론 1회를 뜻하며 환경 5 스텝에 해당한다 — 정책이 action chunk를 예측해 앞의 5개만 실행하고 다시 추론하므로, activation과 GT phase 라벨을 모두 이 시점에서 읽는다. 사람이 주석한 primitive action phase 라벨(instruction당 3 단계)은 학습에 쓰지 않고 평가에만 사용한다. Detector 학습에 사용된 데이터

Clustering. raw activation(1536차원)을 그대로 오토 인코더(AE, latent 16)로 압축한 뒤 KMeans로 스텝 단위 clustering한다(instruction별 분석은 $k=8$, 전체 통합 분석은 $k=24$). Cluster center는 train split에서만 만들고 평가 split의 스텝은 k-NN($k=15$)으로 분류한다. 재군집화가 필요 없으므로 추론 중 스텝 단위 online 관독이 가능하다.

시간 대조군. activation이 phase 정보를 담는지 재려면, 먼저 “시간만 알아도 맞힐 수 있는뿔”을 빼야 한다. 조작 task의 phase는 대체로 순서대로 진행되므로 activation을 전혀 보지 않고 경과 시간만 보는 관독기도 phase를 어느 정도 맞힌다. 이를 시간 대조군(clock)으로 정의한다: 에피소드 내 상대 진행도 t/T 를 발견된 군집 수와 같은 개수의 구간으로 등분위 분할해 얻은 상태열이다. 상태 수를 맞추는 이유는 상태 수가 많을수록 상호정보량(MI)이 자동으로 커지기 때문이며, 이렇게 해야 같은 해상도에서 비교된다.

지표. 군집열이 담는 phase 정보는 MI(군집; GT phase)로 재고, 판정 기준은 시간 대조군을 뺀 $\text{margin} = I(\text{군집}; \text{GT}) - I(\text{clock}; \text{GT})$ 을 쓴다. margin은 activation이 단순 시간 정보를 넘어 추가로 담고 있는 phase 정보량[bits]을 뜻한다. 각 군집이 특정 phase에 얼마나 국한되는지는 순도(purity)로, phase 경계와의 정렬은 전환 위치를 에피소드 내에서 무작위로 혹은 300회 대비 z-score로 잰다.

III 결과

activation phase cluster는 사람이 정의한 phase보다 세밀하다(그림 2a). 발견된 상태의 평균 구간 길이는 GT phase보다 짧다($k=24$ 에서 4.3 vs 16.1 스텝, 전환율 18.4% vs 5.0%; 데이터셋 B에서도 6.0 vs 18.1 스텝). 이 전환은 고정 리듬 아티팩트가 아니고(전환 위치의 주기성 없음), 전환 시점이 GT 경계와 정렬되지도 않는다($z -1.0 \sim +0.5$ — 우연 수준). 그러나 발견 상태를 GT phase 단위로 병합하면 경계 정렬이 유의해진다($z +1.3 \sim +4.9$). 즉 activation은 GT phase를 그대로 재현하는 것이 아니라 그보다 잘게 쪼개

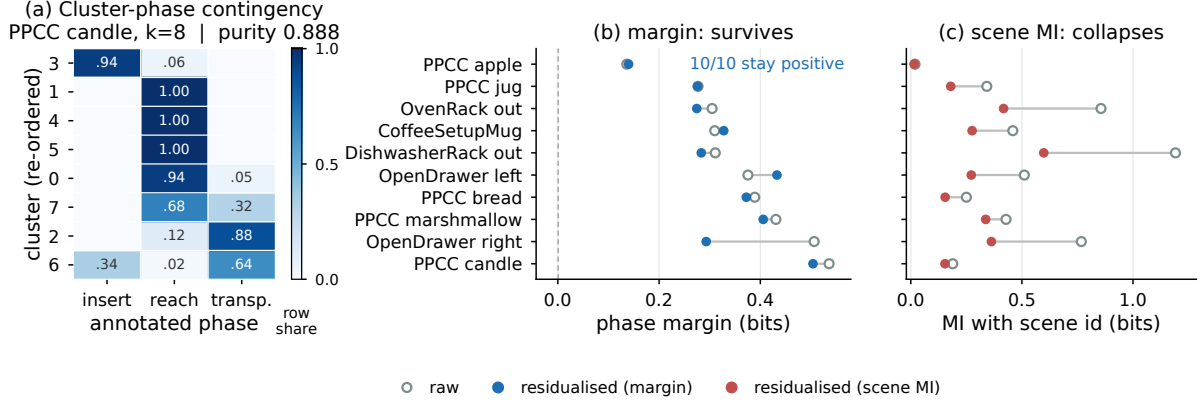


Figure 1: (a) 군집×GT 단계 contingency(PPCC candle, k=8): 행을 최빈 단계로 정렬하면 블록-대각 구조 — 각 군집이 특정 단계에 국한됨(순도 0.910). (b) 장면 잔차화 후에도 단계 margin은 유지되고(9/9 양수) (c) 장면 MI는 붕괴한다.

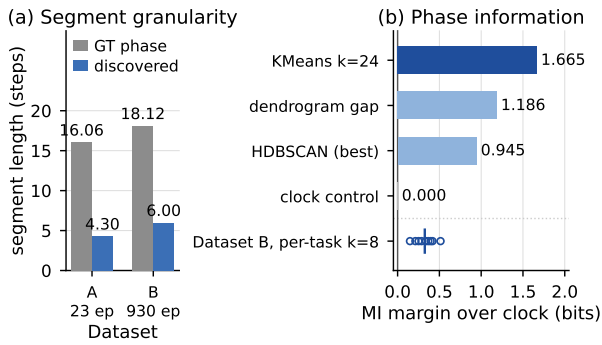


Figure 2: (a) 발견 상태의 구간 길이는 GT 단계의 약 1/4~1/3. (b) 시간 대조군 대비 MI margin: 고정 k KMeans가 자율 k 방식보다 높다(데이터셋 A, k=24 기준 +1.665 bits).

다. 세밀함의 배수는 군집 수 k 에 따라 달라지지만, 세밀해진다는 사실 자체는 k 를 GT phase 수(3~6)에 맞춰도 남는다: $k=8$ 에서도 구간 길이는 GT보다 짧게 유지되고(10개 중 9개 instruction), 에피소드당 전환 수가 상태를 한 번씩만 지나는 단조 통과와 상한($k-1=7$ 회)을 넘는 instruction이 절반이다(최대 22회) — 상태가 재방문되는 구조로, k 를 키워서 만들 수 있는 모양이 아니다. 반대로 한 instruction(OvenRack)은 같은 $k=8$ 에서 GT보다 굵게 병합되어, 해상도는 k 가 아니라 activation이 정한다.

각 군집은 특정 단계에 국한된다(그림 1a, 그림 2b). 한 단계에서 나타난 군집은 다른 단계에서 잘 나타나지 않는다: 군집→단계 순도 0.80, MI 2.17 bits(상한 2.82), 시간 대조군 margin +1.665 bits. 군집이 최빈 단계 밖에서 나타나는 비율(off-phase rate)의 군집별 중앙값은 instruction 중앙값 0.22이며, 순수한 instruction에서는 0.001~0.06까지 내려간다(넓게 걸치는 것도 있어 최대 0.47 — instruction별 편차가 있다). 단계 하나는 평균 2~3개 군집으로 세분되어(단계당 군집 수 1.3~2.7), 대응은 1:1이 아니라 다대일이다 — 이것이 위의 “세밀한 하위 분할”과 순도가 공존하는 이유다. 비디오 행동 분절에서 과분할은 통상 별점이지만, 여기서 세분은 phase-순수한 다대일 대응이며 그 판정 근거는 순도·MI·margin이지 경계 F1이 아니다.

단계 정보는 장면 암기로 환원되지 않는다(그림 1b). 활성화에는 장면(scene) 정보도 섞여 있다(instruction 별 mi_scene 0.2~1.0 bits). 그러나 instruction 내 scene별 평균을 제거한 잔차에서 장면 정보는 대부분 사라지는 반면(mi_scene 중앙값 0.54→0.24 bits) phase margin은 소표본 퇴화 1개를 제외한 9개 instruction 전부에서 양수로 유지된다(중앙값 +0.34→+0.28; 3개는 오히려 상승).

구조는 모델·데이터를 바꿔도 재현된다. 같은 데이터에서 구조가 다른 두 압축 모델(AE, SAE)이 같은 스텝에서 전환하고($F1$ 0.66~0.75, 무작위 0.51~0.53, z +8.8~+13.5), 독립 재수집 데이터에서 새로 학습해도 같은 성질이 나타난다(seed 간 $F1$ 0.55~0.58, z +4.3~+5.1). denoising 슬롯·장면·시드는 에피소드 내 상수라 전환의 원인이 될 수 없다.

세밀 단위는 하류 분석에서도 유용하다. 성공/실패 분리를 단계 조건부로 쪼갤 때, 발견 상태(instruction 별 $k=8$)가 GT phase보다 높은 분리도를 주는 instruction이 9개 중 4개였다(동급 1, 열세 1; 예: OpenDrawer AUROC 0.93/z4.5 vs GT 0.94/z2.3). 이는 탐색적 관찰이나, 세밀 단위가 사람 정의 단계의 단순 잡음이 아님을 시사한다.

IV 한계와 결론

GT 라벨 두 벌은 같은 주석 체계의 두 버전이라 “어떤 단계 정의와도 어긋난다”는 주장은 할 수 없고, 전환 경계 자체는 GT와 정렬되지 않으므로 단계 판독의 근거는 군집 정체성(MI·순도)이지 경계 검출이 아니다. 길이 1의 짧은 상태가 구간의 31~37%를 차지하며 그 의미는 미검증이다.

결론: VLA 활성화에는 사람의 primitive 단계 정의보다 세밀하지만 단계 정합적인 상태 구조가 재현 가능하게 존재하며, k -NN 라벨 전이로 추론 중 온라인 판독이 가능하다. 즉 모델이 지금 무슨 작업을 진행 중이라고 표상하는지를 외부 관측 없이 내부에서 스텝 단위로 읽을 수 있으며, 이는 VLA 행동의 설명 신호이자 단계 조건부 개입과 실패 감지의 조건 신호가 된다.

References

- [1] NVIDIA GEAR Team, “GR00T N1: An Open Foundation Model for Generalist Humanoid Robots,” arXiv:2503.14734, 2025.
- [2] S. Nasiriany et al., “RoboCasa: Large-Scale Simulation of Everyday Tasks for Generalist Robots,” *RSS*, 2024.
- [3] A. Swann et al., “Sparse Autoencoders Reveal Interpretable and Steerable Features in VLA Models,” arXiv:2603.19183, 2026.
- [4] H. Buurmeijer et al., “Observing and Controlling Features in Vision-Language-Action Models,” arXiv:2603.05487, 2026.
- [5] X. Jin et al., “Event-Grounded Sparse Autoencoders for Vision-Language-Action Policies,” arXiv:2605.17204, 2026.
- [6] B. Häon et al., “Mechanistic Interpretability for Steering Vision-Language-Action Models,” *CoRL*, 2025.
- [7] Q. Gu et al., “SAFE: Multitask Failure Detection for Vision-Language-Action Models,” *NeurIPS*, 2025.
- [8] W. Wan et al., “LOTUS: Continual Imitation Learning for Robot Manipulation Through Unsupervised Skill Discovery,” *ICRA*, 2024.

- [9] R. S. Sutton, D. Precup, and S. Singh, “Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning,” *Artificial Intelligence*, 112:181–211, 1999.